

Smart Automated Plant Watering System for Disabled Individuals Using IoT

Aung Khant Win (2306210008)

Naing Min Khant (2209230006)

Kyaw Wai Yan San (2209220003)

Kaung San Thar (2211020004)

Information Technology, Stamford University

Section – 2

ITE233 : Introduction to Internet Of Things

Lecturer – Atikom Srivallop

Abstract

This project aims to assist disabled individuals who love plants and flowers by providing an automated and remote-controlled plant watering system. The system is built using an ESP8266 (NodeMCU), a soil moisture sensor, a relay-controlled water pump, and an LCD display, with Blynk integration for remote monitoring and control. The soil moisture sensor continuously checks the soil's moisture level, and based on predefined thresholds, the system automatically activates or deactivates the water pump. Additionally, users can manually control the watering process via the Blynk app, making it accessible for those with mobility challenges. The LCD display provides real-time updates on soil moisture and motor status. By automating the watering process, this project ensures that plants receive the necessary care without requiring physical effort, promoting independence and convenience for disabled plant lovers.

Table of contents

Abstract.....	2
Table of contents.....	2
Introduction.....	3
Feasibility Study.....	4
1. Technical Feasibility.....	4
2. Economic Feasibility.....	4
3. Operational Feasibility.....	4
4. Social and Environmental Feasibility.....	4
Hardware Components.....	5
Software Tools.....	5
Logical Flowchart of the System.....	6
Hardware setup.....	7
Wire Connections.....	8
Coding.....	8
Code Explanations.....	11
1. Blynk & WiFi Credentials.....	11
2. Define Pins & WiFi Details.....	12
3. System State Variables.....	12
4. Read Soil Moisture Percentage.....	12
5. Blynk Mode Switching (Virtual Pin V3).....	13
6. Manual Motor Control via Blynk (Virtual Pin V1).....	13
7. Send Notifications on Motor State Change.....	13
8. Setup Function.....	14

Step 1: Keep Blynk Running.....	15
Step 2: Read Soil Moisture & Send to Blynk.....	15
Step 3: Track Previous Motor State.....	15
Step 4: Auto Mode - Motor Control Based on Moisture.....	16
Step 5: Send Notification Only If Motor State Changes.....	16
Step 6: Update LCD with Moisture & Motor Status.....	17
Step 7: Delay Before Next Cycle.....	17
Loop execution table.....	18
Structured chart of the program.....	18
Overall flow chart.....	19
Blynk console.....	20
Data streams and event notifications.....	20
Notification results.....	21
Conclusion.....	22
Summary of Project Outcomes.....	22
Future Improvements and Scalability.....	22
References.....	23

Introduction

Gardening is a therapeutic and rewarding activity, but it can be challenging for individuals with disabilities, especially when it comes to tasks like watering plants. This project aims to address this issue by developing a Smart Automated Plant Watering System that enables disabled individuals to care for their plants effortlessly.

The system is built using ESP8266 (NodeMCU), a soil moisture sensor, a relay-controlled water pump, an LCD display, and Blynk integration for remote monitoring and control. The soil moisture sensor continuously checks the moisture level of the soil, and based on predefined thresholds, the system automatically activates or deactivates the water pump. Users also have the option to manually control the watering process via the Blynk mobile app, providing flexibility and ease of use.

With real-time data displayed on an LCD screen and automatic or manual control through a smartphone, this system eliminates the need for physical effort, making gardening more accessible and convenient for disabled individuals. By leveraging IoT technology, this project ensures that plants receive proper care while enhancing the independence and well-being of plant lovers with mobility challenges.

Feasibility Study

The Smart Automated Plant Watering System is designed to assist disabled individuals in caring for their plants with minimal effort. To evaluate the project's viability, a feasibility study is conducted based on four key aspects: technical, economic, operational, and social feasibility.

1. Technical Feasibility

The system utilizes ESP8266 (NodeMCU) as the core microcontroller, which is cost-effective and supports WiFi connectivity for IoT applications. Components such as the soil moisture sensor, relay module, water pump, LCD display, and Blynk platform are readily available and compatible with the system. The project is technically feasible as it involves basic electronics, simple circuit connections, and well-documented software tools, making it easy to implement and maintain.

2. Economic Feasibility

The project is cost-effective compared to commercial automated watering systems. The estimated cost of components remains affordable, and since the system reduces water wastage by optimizing watering schedules, it leads to long-term cost savings. Additionally, open-source platforms like Blynk eliminate the need for expensive cloud services, making the project economically sustainable.

3. Operational Feasibility

The system is designed to be user-friendly and fully automated, requiring little to no physical effort from the user. The Blynk mobile app allows easy remote control, making it accessible for disabled individuals. The system also provides real-time feedback on soil moisture and motor status, ensuring that users can monitor their plants efficiently. With auto and manual modes, the system adapts to different user preferences, making it practical for daily use.

4. Social and Environmental Feasibility

This project promotes inclusivity and independence by empowering disabled individuals to manage plant watering effortlessly. It enhances their quality of life by enabling them to engage in gardening without physical limitations. Additionally, the system optimizes water usage, preventing overwatering and conserving water resources, making it environmentally sustainable.

Hardware Components

Component	Description	Function
ESP8266 (NodeMCU)	WiFi-enabled microcontroller	Processes data, connects to Blynk, and controls the relay
Soil Moisture Sensor	Measures soil moisture levels	Sends moisture data to ESP8266 for automatic watering decisions
Relay Module (5V, 1-Channel)	Electronic switch	Controls the ON/OFF state of the water pump based on ESP8266 signals
Water Pump & Mini Motor (9V)	Small DC water pump	Pumps water to plants when soil is dry
LCD Display (I2C 16x2)	16-character, 2-row display with I2C module	Displays real-time soil moisture percentage and motor status
Power Supply (9V Battery / Adapter)	Provides electrical power	Powers ESP8266, relay, and motor
Jumper Wires	Electrical wires for connections	Connects all components on a breadboard or PCB
Breadboard	Prototyping board for connections	Used for testing circuit connections before soldering

Software Tools

Software Tool	Description	Function
Arduino IDE	Open-source software for coding and uploading programs to microcontrollers	Used to write, compile, and upload code to the ESP8266 (NodeMCU)
Blynk IoT Platform	Cloud-based IoT application for remote monitoring and control	Allows users to control the water pump and monitor soil moisture levels remotely
ESP8266WiFi Library	WiFi library for ESP8266	Enables the ESP8266 to connect to WiFi for internet-based communication
BlynkSimpleEsp8266 Library	Blynk library for ESP8266	Provides communication between the ESP8266 and the Blynk IoT platform
Wire Library	Library for I2C communication	Facilitates communication between ESP8266 and the LCD module
LiquidCrystal_PCF8574 Library	Library for I2C LCD display	Controls the 16x2 LCD screen for displaying moisture level and motor status

Logical Flowchart of the System

Step 1: System Initialization

- Start the ESP8266 microcontroller.
- Connect to WiFi and Blynk Cloud.
- Display system startup message on the LCD.

Step 2: Read Soil Moisture Sensor

- Measure soil moisture percentage.
- Display moisture value on LCD and Blynk App.

Step 3: Decision-Making (Auto Mode)

- If Auto Mode is ON:
- If moisture is too low (e.g., 0-30%), turn ON the water pump.
- If moisture reaches the required level (e.g., $\geq 80\%$), turn OFF the pump.

Step 4: Manual Mode (User Control via Blynk App)

- If Manual Mode is ON, allow users to manually turn the pump ON/OFF using the Blynk App.

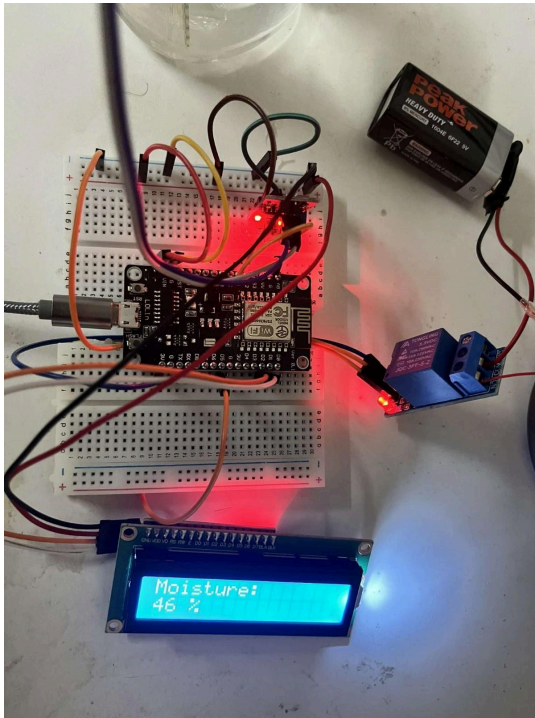
Step 5: Notifications & Updates

- Update the Blynk App with current soil moisture and motor status.
- If motor state changes, send a notification to the user.
- Update LCD display accordingly.

Step 6: Repeat the Process

- Continuously monitor moisture levels and process inputs.
- Maintain real-time control and updates.

Hardware setup



Wire Connections

Component	ESP8266 (NodeMCU) Pin	Other Connections
Soil Moisture Sensor	VCC → 3.3V	(Anode: Positive)
	GND → GND	(Cathode: Negative)
	Signal (AO) → A0	Reads soil moisture
Relay Module	VCC → 3.3V	Power for relay
	GND → GND	Ground connection
	IN (Signal Pin) → D5	Controls relay (motor ON/OFF)
Mini Water Pump	+ (Anode) → Relay NO (Normally Open)	Controlled by relay
	- (Cathode) → GND	Negative terminal
I2C LCD (16x2)	VCC → 5V	Power for LCD
	GND → GND	Ground connection
	SDA → D2	I2C Data
	SCL → D1	I2C Clock
9V Battery (Motor Power)	+ (Anode) → Relay COM (Common)	Provides power
	- (Cathode) → GND	Shared ground

Coding

```
#define BLYNK_TEMPLATE_ID "TMPL6wJ01HRPg"
#define BLYNK_TEMPLATE_NAME "plant watering"
#define BLYNK_AUTH_TOKEN "L-__b906DrZetZ-ZNkRfTb_KNEW2qrGT"

#include <Wire.h>
#include <LiquidCrystal_PCF8574.h>
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

// LCD setup
LiquidCrystal_PCF8574 lcd(0x27);

// Soil moisture sensor pin
const int soilMoisturePin = A0;

// Relay pin (Motor Control)
const int relayPin = D5;
```



```

// WiFi credentials
char auth[] = "L-__b906DrZetZ-ZNkRfTb_KNEW2qrGT";
char ssid[] = "iPhone";
char pass[] = "11111110";

// Motor state and mode
bool motorState = false;
bool autoMode = true; // Default to Auto Mode

// Function to read soil moisture percentage
int readMoisture() {
    int sensorValue = analogRead(soilMoisturePin);
    int percentage = map(sensorValue, 1023, 0, 0, 100); // Adjust mapping
    if necessary
        return percentage;
}

// Function to handle mode switch (V3) from Blynk
BLYNK_WRITE(V3) {
    autoMode = param.asInt(); // 1 = Auto Mode, 0 = Manual Mode
}

// Function to handle manual motor control (V1) from Blynk
BLYNK_WRITE(V1) {
    if (!autoMode) { // Only allow manual control when in Manual Mode
        bool newState = param.asInt(); // 1 = ON, 0 = OFF
        if (newState != motorState) { // Only send notification if state
            changes
            motorState = newState;
            digitalWrite(relayPin, motorState ? LOW : HIGH); // Control the
            motor
            sendMotorNotification(motorState);
        }
    }
}

// Function to send notifications when motor turns ON or OFF
void sendMotorNotification(bool state) {
    if (state) {
        Blynk.logEvent("motor_on", "Motor has been turned ON");
    } else {
        Blynk.logEvent("motor_off", "Motor has been turned OFF");
    }
}

void setup() {
    Serial.begin(115200);

    // Initialize LCD

```

```

lcd.begin(16, 2);
lcd.setBacklight(255);

// Display welcome message
lcd.setCursor(0, 0);
lcd.print("Let's water");
lcd.setCursor(0, 1);
lcd.print("System Start...");
delay(2000);
lcd.clear();

// Initialize relay
pinMode(relayPin, OUTPUT);
digitalWrite(relayPin, HIGH); // Ensure motor is off at startup

// Connect to WiFi and Blynk
WiFi.begin(ssid, pass);
Blynk.begin(auth, ssid, pass);
}

void loop() {
  Blynk.run();

  int moisturePercentage = readMoisture();
  Blynk.virtualWrite(V2, moisturePercentage); // Send soil moisture
  level to Blynk

  static bool lastMotorState = false; // Track previous motor state

  // Auto Mode motor control
  if (autoMode) {
    if (moisturePercentage == 0 && !motorState) { // If soil is dry
and motor is OFF
      motorState = true;
      digitalWrite(relayPin, LOW); // Turn ON the motor
    }
    else if (moisturePercentage >= 80 && motorState) { // If soil is
wet and motor is ON
      motorState = false;
      digitalWrite(relayPin, HIGH); // Turn OFF the motor
    }
  }

  // Send notification only if motor state changes
  if (motorState != lastMotorState) {
    sendMotorNotification(motorState);
    lastMotorState = motorState;
  }
}

```

```
// Display data on LCD
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Moisture: ");
lcd.print(moisturePercentage);
lcd.print("%");

lcd.setCursor(0, 1);
lcd.print("Motor: ");
lcd.print(motorState ? "ON" : "OFF");

delay(1000); // Update every second}
```

Code Explanations

1. Blynk & WiFi Credentials

```
#define BLYNK_TEMPLATE_ID "TMPL6wJ01HRPg"
#define BLYNK_TEMPLATE_NAME "plant watering"
#define BLYNK_AUTH_TOKEN "L-__b906DrZetZ-ZNkRfTb_KNEW2qrGT"

#include <Wire.h>
#include <LiquidCrystal_PCF8574.h>
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

// LCD setup
LiquidCrystal_PCF8574 lcd(0x27);
```

- Defines Blynk authentication details.
- Includes necessary libraries for ESP8266, LCD, and Blynk.
- LCD Address 0x27 is set for I2C communication.

2. Define Pins & WiFi Details

```
const int soilMoisturePin = A0; // Soil Moisture Sensor (Analog Input)
const int relayPin = D5;        // Relay Control Pin (Motor)

char auth[] = "L-__b906DrZetZ-ZNkRfTb_KNEW2qrGT"; // Blynk Token
char ssid[] = "iPhone"; // WiFi SSID
char pass[] = "11111110"; // WiFi Password
```

- Soil moisture sensor is connected to A0.
- Relay (Motor Control) is connected to D5.
- WiFi credentials for ESP8266 to connect to the internet.

3. System State Variables

```
bool motorState = false; // Tracks if motor is ON or OFF
bool autoMode = true;    // Default to Auto Mode
```

- `motorState` → Tracks if the motor is running.
- `autoMode` → Controls whether the system is in automatic or manual mode.

4. Read Soil Moisture Percentage

```
int readMoisture() {
    int sensorValue = analogRead(soilMoisturePin);
    int percentage = map(sensorValue, 1023, 0, 0, 100); // Convert to %
    return percentage;
}
```

- Reads analog value from A0.
- Converts it into a percentage (0-100%).

5. Blynk Mode Switching (Virtual Pin V3)

```
BLYNK_WRITE(V3) {  
  autoMode = param.asInt(); // 1 = Auto Mode, 0 = Manual Mode  
}
```

- Allows the user to toggle between Auto & Manual mode in Blynk.

6. Manual Motor Control via Blynk (Virtual Pin V1)

```
BLYNK_WRITE(V1) {  
  if (!autoMode) { // Only allow manual control when in Manual Mode  
    bool newState = param.asInt(); // 1 = ON, 0 = OFF  
    if (newState != motorState) { // Only send notification if state changes  
      motorState = newState;  
      digitalWrite(relayPin, motorState ? LOW : HIGH); // Motor ON/OFF  
      sendMotorNotification(motorState);  
    }  
  }  
}
```

- **Only works if Auto Mode is OFF.**
- **Turns motor ON/OFF** when manually toggled.
- **Sends a notification** when the motor state changes.

7. Send Notifications on Motor State Change

```
void sendMotorNotification(bool state) {  
  if (state) {  
    Blynk.logEvent("motor_on", "Motor has been turned ON");  
  } else {  
    Blynk.logEvent("motor_off", "Motor has been turned OFF");  
  }  
}
```

- Sends a notification when the motor turns ON or OFF.

8. Setup Function

```
// Connect to WiFi and Blynk  
WiFi.begin(ssid, pass);  
Blynk.begin(auth, ssid, pass);  
}
```

```
void setup() {  
  Serial.begin(115200);  
  
  // Initialize LCD  
  lcd.begin(16, 2);  
  lcd.setBacklight(255);  
  lcd.setCursor(0, 0);  
  lcd.print("Let's water");  
  lcd.setCursor(0, 1);  
  lcd.print("System start...");  
  delay(2000);  
  lcd.clear();  
  
  // Initialize relay  
  pinMode(relayPin, OUTPUT);  
  digitalWrite(relayPin, HIGH); // Ensure motor is OFF at startup  
}
```

- LCD displays a startup message.
- Relay is initialized (motor OFF at start).
- Connects ESP8266 to WiFi & Blynk.

Loop Function

Step1:Keep Blynk Running

```
Blynk.run(); // Run Blynk
```

Step 2: Read Soil Moisture & Send to Blynk

```
int moisturePercentage = readMoisture();  
Blynk.virtualWrite(V2, moisturePercentage); // Send soil moisture to Blynk
```

- Calls readMoisture() to measure soil moisture from the sensor.
- Sends the moisture value to Virtual Pin V2 in Blynk for monitoring.

Step 3: Track Previous Motor State

```
static bool lastMotorState = false; // Track previous motor state
```

- lastMotorState remembers the previous motor state.
- This helps in sending notifications only when the state changes

Step 4: Auto Mode - Motor Control Based on Moisture

```
if (autoMode) {  
  if (moisturePercentage == 0 && !motorState) { // If soil is dry and motor is OFF  
    motorState = true;  
    digitalWrite(relayPin, LOW); // Turn ON the motor  
  }  
  else if (moisturePercentage >= 80 && motorState) { // If soil is wet and motor  
    motorState = false;  
    digitalWrite(relayPin, HIGH); // Turn OFF the motor  
  }  
}
```

- Auto Mode is active (controlled by V3 in Blynk).
- If moisture is 0% (dry soil) → Turn ON motor (pump starts watering)
- If moisture is 80%+ (wet soil) → Turn OFF motor (stop watering).

Step 5: Send Notification Only If Motor State Changes

```
if (motorState != lastMotorState) {  
  sendMotorNotification(motorState);  
  lastMotorState = motorState;  
}
```

- Checks if motor state changed (from ON → OFF or OFF → ON).
- If changed, send a notification via Blynk.
- Updates lastMotorState to track the last known motor state.

Step 6: Update LCD with Moisture & Motor Status

```
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Moisture: ");  
lcd.print(moisturePercentage);  
lcd.print("%");  
  
lcd.setCursor(0, 1);  
lcd.print("Motor: ");  
lcd.print(motorState ? "ON" : "OFF");
```

- Clears the LCD before updating (prevents overlapping text).
- First line: Displays soil moisture as a percentage.
- Second line: Displays whether the motor is ON or OFF.

Step 7: Delay Before Next Cycle

```
delay(1000); // Update every second
```

- Waits for 1 second before repeating the loop.
- This prevents the system from refreshing too quickly.

Loop execution table

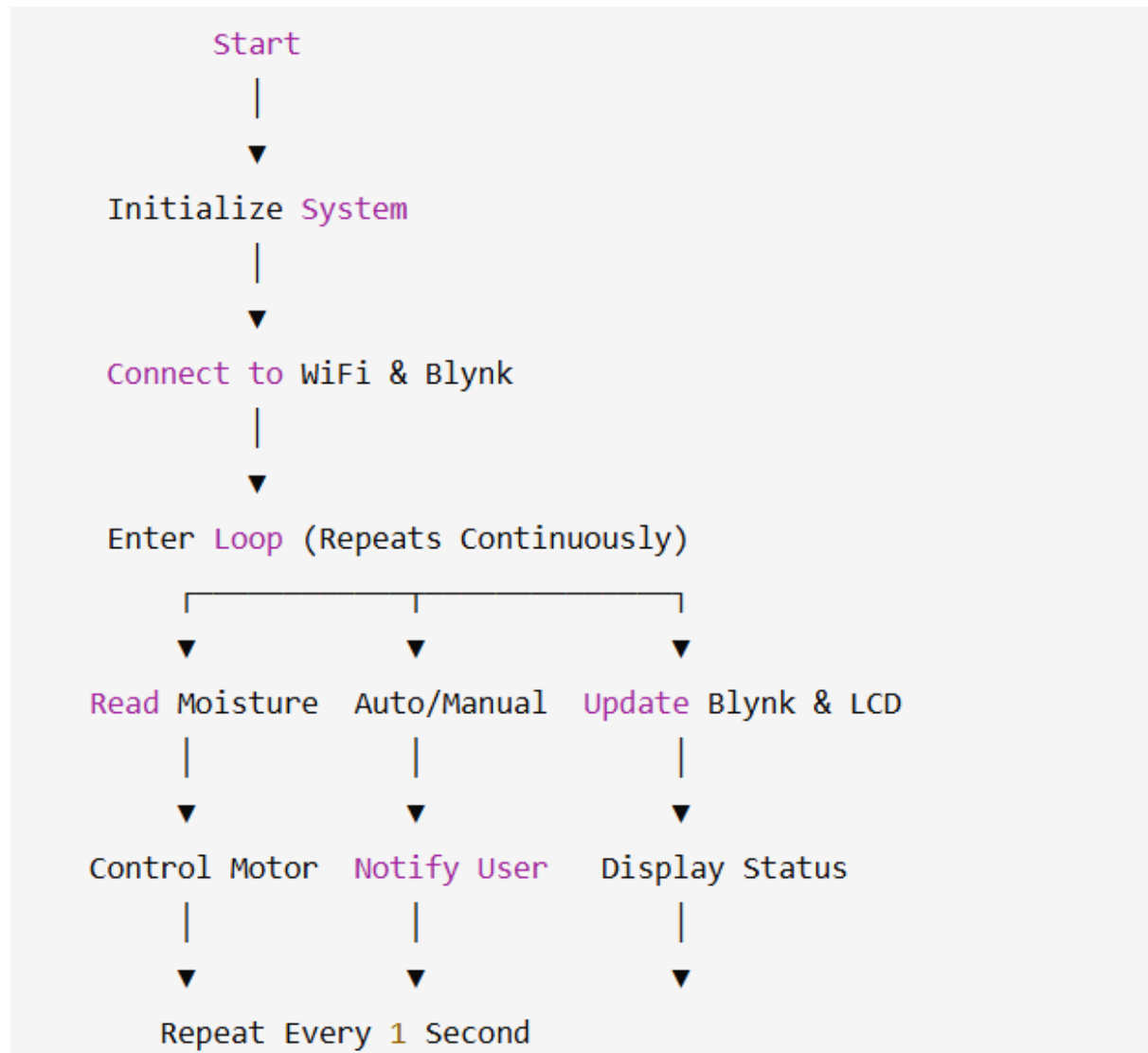
Moisture %	Motor state(auto)	LCD output	Blynk update
0%	ON(watering)	Moisture: 0% Motor: ON	Sends 0% to V2
50%	ON(still watering)	Moisture: 50% Motor: ON	Sends 50% to V2
80%	OFF(stop watering)	Moisture: 80% Motor: OFF	Sends 80% to V2

Structured chart of the program

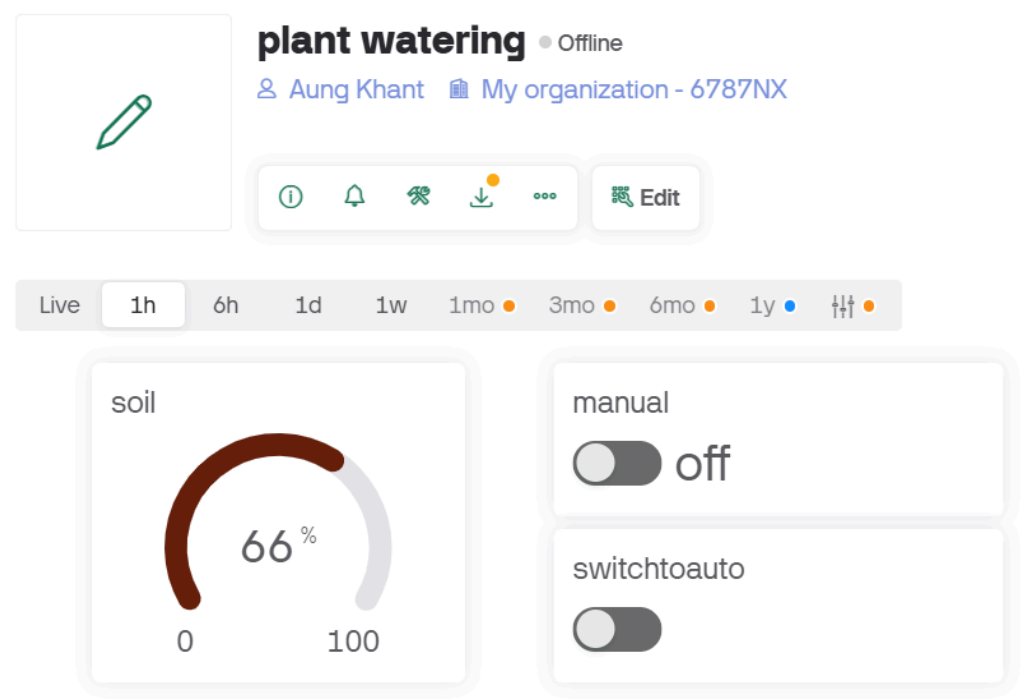
The program is designed to automate plant watering using an ESP8266 (NodeMCU), a soil moisture sensor, a relay-controlled water pump, and Blynk for remote monitoring and control. The structure consists of several functions that work together to read soil moisture, control the motor, and update both the Blynk app and an LCD display.

Function	Purpose
<code>setup()</code>	Initializes LCD, WiFi, Blynk, and relay setup.
<code>loop()</code>	Runs continuously, reads sensor data, updates Blynk, and controls the motor.
<code>readMoisture()</code>	Reads the soil moisture sensor and converts it to percentage.
<code>BLYNK_WRITE(V3)</code>	Reads Auto/Manual mode switch from Blynk (V3).
<code>BLYNK_WRITE(V1)</code>	Allows manual ON/OFF control of the motor via Blynk (V1).
<code>sendMotorNotification(state)</code>	Sends notifications when motor turns ON/OFF.

Overall flow chart



Blynk console



plant watering: motorOff Inbox x



Blynk <robot@blynk.cloud>
to me ▾

motorOff

Motor has been turned OFF

[Open in the app](#) | [Mute notifications](#)

--

Date: Saturday, February 22, 2025, 5:06:42 PM Indochina Time

Device name: [plant watering](#)

Organization: [My organization - 6787NX](#)

Template: plant watering

Owner: [aungkhantwin095@gmail.com](#)

↩ Reply

➦ Forward



Notification results

plant watering: motoron Inbox x



Blynk <robot@blynk.cloud>
to me ▾

motoron

Motor has been turned ON

[Open in the app](#) | [Mute notifications](#)

--

Date: Saturday, February 22, 2025, 5:06:32 PM Indochina Time

Device name: [plant watering](#)

Organization: [My organization - 6787NX](#)

Template: plant watering

Owner: [aungkhantwin095@gmail.com](#)

↩ Reply

➦ Forward



Conclusion

Summary of Project Outcomes

The Smart Automated Plant Watering System successfully provides an efficient and accessible solution for disabled individuals who love gardening. By integrating ESP8266, a soil moisture sensor, a relay-controlled water pump, and Blynk IoT technology, the system can automatically monitor and manage plant watering without requiring manual effort.

The system operates in two modes:

Auto Mode: The motor turns ON/OFF automatically based on soil moisture levels.

Manual Mode: Users can control the watering process remotely via the Blynk app.

The LCD display provides real-time updates, and Blynk notifications alert users when the pump is activated or deactivated. This ensures efficient water usage while enhancing the independence and convenience of disabled users.

Future Improvements and Scalability

While the project is functional, several improvements can enhance its performance and usability:

- Battery Optimization & Solar Power
- Implementing solar power for the ESP8266 and water pump would improve sustainability.
- Adding deep sleep mode for ESP8266 can reduce power consumption.
- Multiple Plant Monitoring
- Expanding the system to monitor multiple plants by using multiple soil moisture sensors.
- Enabling zone-based watering for different plant types.
- Machine Learning for Smart Watering
- Using AI-based predictions to optimize watering schedules based on weather conditions.
- Implementing a learning algorithm to adjust watering based on past moisture trends.
- Integration with Voice Assistants
- Enabling voice commands via Google Assistant or Alexa for hands-free control.
- Mobile App Enhancements

- Adding historical moisture data analysis in Blynk for better insights.
- Implementing custom watering schedules based on plant needs.
- Offline Functionality
- Enabling automatic watering even when WiFi is unavailable by storing conditions locally.

By implementing these enhancements, the system can become more scalable, energy-efficient, and intelligent, making plant care even more accessible for disabled individuals while promoting water conservation and smart agriculture.

References

Elecrow. (2018, November 12). Revolutionize Your Garden with Elecrow Arduino

Automatic Watering Kit. Retrieved February 23, 2025, from Youtu.be website:

<https://youtu.be/3WieNWgikEQ?si=Cf91AMVfa5nDEmnB>

Sayan Electronics. (2023, July 11). How To Make Smart Plant Watering System With

ESP8266 NodeMCU & New Blynk. Retrieved February 23, 2025, from

Youtu.be website: <https://youtu.be/gD4HunCLUmo?si=AyCIJxYmdzDYBmYc>

SriTu Hobby. (2022, July 23). How to make a plant watering system with the

Nodemcu ESP8266 board and the new Blynk update. Retrieved February 23,

2025, from Youtube website:

<https://youtu.be/WLCR0r7rYi8?si=iuqLGBJ6VhjXLoQ3>